UniMind: Calibration-Aware Scheduling and Placement for Quantum-Accelerated AI Middleware

Y. X. Tan

***Abstract*—Quantum-accelerated AI workloads must be scheduled onto physical devices whose calibration changes between runs. A classical scheduler assumes resources are identical and stable; a quantum scheduler faces an array of qubits whose individual noise parameters drift daily. This paper asks: *how should a quantum runtime maintain scheduling decisions when hardware calibration changes over time?* We instrument a 156-qubit superconducting device across three daily calibration snapshots (D0–D2; 2026-08-29 to 08-31) and build UniMind, a middleware that scores, places, executes, and refreshes calibration data in a closed loop. Three findings are established: (1) Calibration validity is temporally bounded. Day-over-day rank correlation is only ρ=0.579, top-10 overlap is 0.25, and a stale top-3 pin is 2.1× worse than a fresh selection within 24 h. We formalize the *Calibration Validity Horizon* T_{valid}: the time window over which a calibration-derived placement remains within fidelity tolerance. (2) Adaptive refresh beats static and periodic policies. A two-line trigger (top-10 Jaccard < 0.5 or $\Delta C > 0.005$) detects staleness at near-zero cost (~ 0.04 ms per scoring pass). Offline threshold sweeps over $\tau_J \in [0.1, 0.9]$ and $\tau_C \in [0.001, 0.02]$ reveal the Pareto tradeoff between refresh rate and fidelity; the measured trigger sits near the knee. (3) Refresh, not placement selection, dominates end-to-end reliability. Fresh pins lift *both* the pinned and free arms by $+33/37$ pp on a 6-qubit angle suite; the pinned vs. free difference is within noise (70.0% vs. 76.7%, p=0.56). A negative result completes the picture: re-fitting the 7-dim layout utility $J(G)$ on real device labels collapses its out-of-sample Spearman from ρ=0.818 to 0.13 (p=0.36), because the simulator proxy over-penalizes large layouts that a refresh-pinned device does not. We give the validity model, the measured numbers, the adaptive mechanism, and the explicit negative results.**

***Index Terms*—quantum computing middleware, calibration-aware scheduling, calibration validity horizon, adaptive refresh, qubit placement, calibration drift, resource management, quantum-cloud runtime, reliability composition.**

I. Introduction

EMBEDDED AI is increasingly offloaded to accelerator hardware, and quantum processors are the newest such accelerator. Their software stack, however, changes the *scheduler's* job in a fundamental way: a quantum circuit's fidelity depends on *which* physical qubit executes it and *when* that qubit was last calibrated [1], [2], [5], [6]. A classical scheduler assumes resources are identical and stable; a quantum scheduler faces an array of hundreds of qubits whose individual noise parameters move between runs [10], [13], [17].

Quantum-machine-learning (QML) frameworks—PennyLane [4], Qiskit [18], TensorFlow Quantum—provide algorithm abstractions but not the *system* layer: dynamic backend discovery, layout selection, and calibration refresh. Applications are assumed, usually silently, to have been placed on "good enough" qubits. On real devices this assumption fails measurably: with *free* transpiler placement an angle-encoding experiment on `ibm_marrakesh` missed the 0.05 fidelity tolerance by up to 0.392, while the *same* circuits on a pinned calibrated layout passed everywhere (max. deviation 0.028) [25]. Placement is not cosmetic; it is the dominant accuracy term at our test scale.

This paper addresses the temporal dimension of that problem. Placement decisions are made from a calibration snapshot, and snapshots change daily; the scheduler's real questions are "*how old is my snapshot*" and "*should I refresh*" as much as "*where to place*". We define the **Calibration Validity Horizon** T_{valid}: the maximum age of a calibration snapshot such that the resulting placement remains within fidelity tolerance. With T_{valid} small (hours, not days), the runtime must refresh frequently; with T_{valid} large, refresh is wasteful. We measure T_{valid}'s lower bound on a 156-qubit device across three daily snapshots, design an adaptive refresh trigger that detects staleness at near-zero cost, and show that refresh—not placement selection—is where reliability engineering should invest.

This paper makes three contributions, all measured on a 156-qubit `ibm_marrakesh` device across three daily calibration snapshots (D0–D2; 2026-08-29 to 08-31):

1) **Temporal validity analysis.** We quantify day-over-day calibration drift (ρ=0.579, τ=0.431, top-10 Jaccard 0.25), measure the staleness cost of aged pins (stale top-3 is 2.1× worse than fresh within 24 h), and formalize the Calibration Validity Horizon T_{valid} as a design primitive for quantum runtimes (§VI).
2) **Adaptive refresh policy.** We design a trigger (Jaccard < 0.5 or $\Delta C > 0.005$) that detects staleness at near-zero cost, run offline threshold sweeps over τ_J and τ_C to reveal the refresh-rate-vs.fidelity Pareto tradeoff, and compare static, periodic, and adaptive policies on the same data (§VI, §VII).
3) **Negative results.** The 7-dim layout utility $J(G)$, trained on a simulator proxy and re-fitted on real device labels, collapses from out-of-sample ρ=0.818 to 0.13 (p=0.36); at k=6 with fresh pins, pinned placement shows no measured advantage over the free transpiler (70.0% vs. 76.7%, p=0.56). We report these as bounds on what placement engineering can deliver without temporal control (§V, §VII).

Scope. We make no claim of quantum advantage, novel quantum algorithms, or a production runtime. We propose T_{valid} as a design primitive; with three daily snapshots we establish its lower bound and demonstrate the adaptive mechanism, but leave

Y. X. Tan is an independent researcher (e-mail: yxtan5022@gmail.com). All experiments are open-source and reproducible from `yxtan5022-maker/unimind-v2`; QPU job IDs and snapshots are recorded in the data directory.

the full multi-day decay curve and cross-device generalization to future work. Every quantitative claim traces to a recorded job or snapshot; where an exploratory result did not replicate (a first run's perfect rank correlation, ρ=1.000, which a fixed-design replication returned as ρ=0.771, p=0.103) we report it with its failure. UniMind also contains an LLM-orchestration layer whose reliability composition we *do* model exactly (§VII-A), but the LLM's quality is a controlled parameter of our experiments, never a claim about real models.

II. Problem and Related Work

A. Formal model

Definition 1 (Calibration-aware scheduling problem): A device is a set of qubits Q and couplers E with a calibration snapshot S_t reporting, per qubit, readout assignment errors $(p_{0|1}, p_{1|0})$, gate errors, and coherence times (T_1, T_2). A job is a logical circuit $\hat{G}$ over k logical qubits. The scheduler chooses a *placement* $\pi : \hat{G} \to Q$ (with $|\pi| = k$, connected in (Q, E) for our chains), a transpiler configuration, and a *snapshot age policy* that decides when S_t is re-fetched. Its objective, subject to placement, snapshot age, and overhead budgets, is per-qubit measurement fidelity $\max_i |Z_i^{\text{emp}} - Z_i^{\text{th}}| \leq \epsilon$ at a reliability target.

The distinctness from classical placement lies in (Q, E, S_t) being jointly non-stationary: π chosen at time t is evaluated at execution time $t' > t$ against a changed $S_{t'}$. The paper's arithmetic is about the size of that violation.

B. Related work

Placement and compilation. Qubit mapping has both heuristic and exact treatments: SABRE [6] and noise-adaptive mappings [5] fold device error into the routing pass; exact synthesis is practical for small circuits [7]; resource-aware timing variants exist for heterogeneous devices [8]. Our contribution is not a new routing algorithm but *who decides*: a middleware layer that computes an `initial_layout` from a score over a live snapshot and composes with any downstream transpiler, plus an explicit staleness contract—aspects the compiler literature treats as "the backend's problem".

Error mitigation. Measurement twirling [11], [12], probabilistic error cancellation, and sparse Pauli–Lindblad models [9], [10] correct errors statistically at shot cost. Our results bound where mitigation is needed: it helps the *broken* qubit and is redundant on the good one (§IV), which argues for calibration-conditional mitigation—a placement decision, not a post-processing one.

Runtime and cloud middleware. Qiskit Runtime [16], Braket [20], and Azure Quantum [21] manage jobs and queue bounds but expose placement as an option, not as a scheduler resource; much of the practical "does this job need mitigation, where, and with which snapshot" knowledge lives in operator runbooks. We know of no published quantified day-scale ranking drift for a 156-qubit device; the public signals closest to it are per-device aggregate fidelities in documentation [17] and in error-rate studies of specific devices [3]. We present the first measurements of the drift and the protocol (trigger, refit-on-real-labels) that a runtime can adopt.

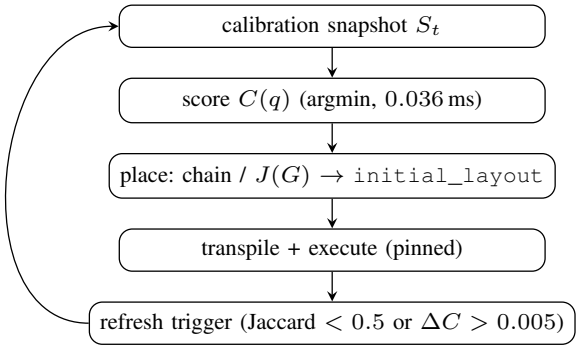

Fig. 1. The UniMind scheduling loop. One scoring pass is sub-millisecond; the refresh trigger decides when the stored snapshot must be re-fetched. ⟨execution → results⟩ is emitted downstream.

III. UniMind Middleware

UniMind is a user-space, multi-backend middleware (Qiskit/Aer, CUDA-Q [19] via Qiskit, IBM Quantum). Its scheduling core is the calibration-aware loop of Fig. 1: one snapshot per calibration cycle, $C(q)$ scoring, chain/utility placement, execution, and a refresh trigger. Two supporting layers are described briefly. The *orchestrator* interprets a task intent into candidate code under sandboxed validation (static analysis, pattern checks, semantic checks) with a deterministic rule-based fallback; its two-parameter quality model (q = per-call correctness, f = unavailability) feeds the reliability calculus of §VII-A. The *Unibit* preprocessing layer derives, from a classical amplitude sequence, the encoding parameters $\theta_i = 2 \arcsin \sqrt{w_i}$ used by every experiment in this paper (sliding-window frequency estimation plus sinc smoothing); the quantum circuit is standard angle (RY) encoding [22], [23], which we use only as an instrument to expose placement and drift.

A. Score

Every qubit is scored by total readout assignment error

$$C(q) \;=\; p_{0|1}(q) + p_{1|0}(q), \tag{1}$$

ties broken by $-\min(T_1, T_2)$. The score is fixed *a priori*, never fitted; it is tested out-of-sample across the entire calibration spectrum (§IV-B) rather than tuned on the data it later predicts.

B. Placement

For a k-qubit circuit we grow a *connected chain*: from the best $C(q)$ qubit, repeatedly add the lowest-C neighbour of the current set (a greedy Prim-style expansion; 0.3 ms at k=3, scales to the full device). For utility-based ranking among candidate layouts,

$$\begin{aligned} J(G) = \alpha E_{\text{readout}} + \beta E_{1\text{q}} + \gamma E_{2\text{q}} + \gamma_{\log} E_{2\text{q\_log}} \\ + \eta_{\text{idle}} E_{\text{idle}} + \delta N_{\text{SWAP}} + \lambda D, \end{aligned} \tag{2}$$

with E_{readout} the mean $C(q)$ of placed qubits, $E_{1\text{q}}$ mean single-qubit gate error, $E_{2\text{q}} = \sum_c p_{\text{cz}}(c)$, $E_{2\text{q\_log}} = \sum_c -\log(1 - p_{\text{cz}}(c))$ (log-odds coupler cost), $E_{\text{idle}} = \text{depth} \cdot \text{mean}(1/T_1 + 1/T_2)$, N_{SWAP} inserted SWAPs, and D transpiled depth;

TABLE I
PLACEMENT-CONTROLLED SWEEP (2026-08-23; 19 WEIGHTS, 8192 SHOTS, 3 JOBS/CELL). DEVIATIONS ARE $\max_w |Z_{\text{emp}} - Z_{\text{th}}|$; TOLERANCE 0.05.

Placement	$C(q)$	Bare	Twirl	Verdict
q98 (best)	0.0044	0.028	0.031	pass
q0 (free)	—	0.026	—	pass
q37 (adversarial)	0.82	0.213	0.144	fail
v1.7 free layout	unrec.	0.392	0.245	fail

features min–max normalized over training points. Section V measures what this utility is worth when its labels come from the real device.

C. Refresh trigger

After each calibration cycle the middleware recomputes the top-10 of $C(q)$ and refreshes the stored snapshot when (i) top-10 Jaccard overlap with the stored snapshot falls below 0.5, or (ii) the stored best qubit's C rises by more than 0.005 absolute. Re-scoring is ~ 0.04 ms—economically free—so staleness is purely an operational-policy failure, quantified next.

IV. PLACEMENT-DOMINATED FIDELITY ON ONE QUBIT

The anchor experiment is a 2×2 design: {best-calibrated qubit, adversarially bad qubit}×{bare, measure-twirled}, on the 19-weight grid $w \in \{0.05, \ldots, 0.95\}$ of the amplitude encoding, 8192 shots, transpiler seed 42, layouts pinned via `initial_layout`, 3 jobs per cell. On the 2026-08-23 snapshot, q98 minimizes C at 0.0044 (T_1=207 μs, T_2=67 μs); q37 is the adversarial anchor (C=0.82).

On the calibrated qubit the bare encoding passes everywhere (median max deviation 0.028; job range 0.020–0.029) and the transfer curve is fit at shot-noise level by $P_{\text{obs}}(1) = 0.004 + 0.989\,w$ (residual RMS 0.0042, below the ≈ 0.0055 shot-noise floor). On q37 the same circuits fail by $\sim 4\times$, and the fitted channel acquires a large offset, $P_{\text{obs}} = 0.121 + 0.855\,w$—the signature of an asymmetric readout channel. Twirling changes nothing significant on the good qubit and only partially rescues the bad one ($0.213 \to 0.144$). Fig. 2 plots the transfer curves. Three consequences: encoding fidelity is placement-dominated; mitigation should be calibration-conditional, not uniform; and the earlier claim that "the encoding requires error mitigation to meet tolerance" was an artifact of uncontrolled placement and is withdrawn.

A. Calibration predictability

Across the whole design, at shot-noise level,

$$P_{\text{obs}}(1) = b + a\,w, \qquad Z_{\text{obs}} = d + cZ_{\text{th}},\ c{=}a,\ d{=}1{-}2b{-}a, \tag{3}$$

with $(a, b){=}(0.99, 0.00)$ on the good qubit vs. $(0.86, 0.12)$ on the bad one; the offset matches the direction and scale of that qubit's readout asymmetry. The one-parameter contraction model $P_{\text{obs}} = 0.5 + \alpha(w - 0.5)$ is the balanced-readout special case $b{=}(1-a)/2$ and is rejected where asymmetry dominates. Matched calibration–noise-model simulation on Aer explains most of observed error on the good qubit (median QPU/model deviation ratio 1.4×): the "standard models underestimate hardware" gap is largely a placement confound, not a model gap.

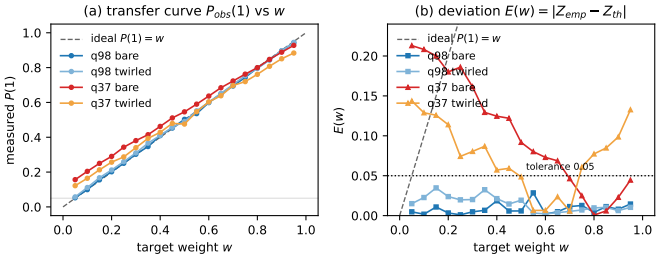

Fig. 2. Placement-dominated encoding fidelity: the bare response on the best-calibrated qubit tracks the ideal line within shot noise; the adversarially chosen q37 compresses the curve toward an offset set by its asymmetric readout.

TABLE II
$C(q)$-STRATIFIED SWEEP (D0 SNAPSHOT; SIX QUBITS × 19 WEIGHTS × 8192 SHOTS). 'PASS' = MAX DEVIATION < 0.05 OVER ALL 19 WEIGHTS; (a, b) FROM EQ. (3).

Qubit	Rank pct.	$C(q)$	max dev	pass	(a, b)
q8	0.0%	0.004	**0.030**	19/19	(0.986, +0.001)
q109	25.2%	0.017	0.081	14/19	(0.933, +0.043)
q105	50.3%	0.031	0.043	19/19	(0.983, +0.018)
q53	75.5%	0.072	0.046	19/19	(0.982, +0.021)
q37	81.9%	0.099	0.146	9/19	(0.887, +0.081)
q27	95.5%	0.346	0.096	13/19	(0.917, +0.049)

B. Stratified validation

From the D0 snapshot all 156 qubits are ranked: q8 is 1/156 (C=0.004), q37 is 128/156 (C=0.10). Six qubits sampled across the ranking run the identical bare 19-weight sweep (Table II). The top qubit passes all 19 weights (0.030 max dev; affine slope degrades along the ranking in the predicted direction: a: 0.986→0.887, offset b: $+0.001 \to +0.081$), but the trend is *not monotone*: q109 at percentile 25.2% already fails, so Spearman ρ=0.771 at n=6 is not significant (exact permutation p=0.103) and a "top-half rule" is unsupported. An earlier exploratory run (first snapshot, 08-22) was perfectly monotone (ρ=1.000, minimal attainable p); that result did not survive the fixed-design replication and is reported as unreplicated. Overhead: 0.036 ms full-156 ranking; pinned transpilation 1.9 ms vs. 1.8 ms free.

C. Multi-qubit routing

Single-qubit $C(q)$ is necessary but not sufficient. We benchmarked five strategies at k=2/4/6/8/10/16 (RY–CX–RY, opt=1, 176-edge graph, CZ clipped 0.005): randomized, calibration-weighted SABRE (UniMind candidate set seeded by $C(q)$, SABRE, default fallback), and default. UniMind reduces SWAPs by 76–100% vs. random and matches default for $k \geq 8$ by fallback; the log-odds coupler cost separates high-error couplers (Random 1.72 vs. UniMind 0.14 at k=8). This is a *system* result about routing cost; whether the chosen layout is *correct* is what §V measures.

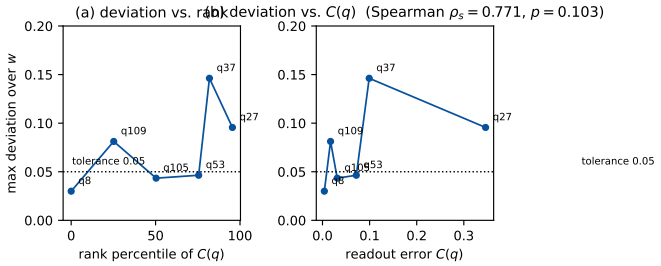

Fig. 3. $C(q)$-guided ranking validation: top-ranked qubits pass, but deviations are not monotone along the ranking (q109 at percentile 25.2% fails), and score vs. deviation gives Spearman 0.771 (p=0.103, n=6).

TABLE III
FITTING $J(G)$ ON SIMULATOR-PROXY VS. REAL-DEVICE LABELS. SAME FEATURE MATRIX AND PROTOCOL; THE $k\geq 9$ LABELS DIFFER. SIZE-OOS = FIT ON $k \leq 8$, EVALUATE $k \geq 9$.

Labels	full fit	LOO	size-OOS
v4 proxy (k=9–18 est-2q)	0.967	0.953	0.818 (p=0.0029)
v5 real (7 measured k)	0.643	0.510	0.127 (p=0.36)
Δ	−0.324	−0.443	−0.691

V. THE $J(G)$ UTILITY: PROXY FIT VS. REAL LABELS

A. Fit protocol

The seven weights of Eq. (2) are fitted by max-Spearman simplex search (Dirichlet+corners sampler, 5 000 samples, seed 42) on n=23 points: six single-qubit real max-dev labels from §IV-B and seventeen multi-qubit points k=2–18 whose label is the simulated two-qubit error E_{2q} of the placed layout under the device error model. Validation: leave-one-out (LOO) and a bootstrap (B=200). A *size out-of-sample* split additionally holds out the entire $k\geq 9$ range: fit on $k\leq 8$ only, evaluate on $k\geq 9$ with train-only min-max scaling. All numbers in Table III use the identical feature matrix; only the labels differ.

B. Result

Trained on the simulator proxy the utility looks decisive: full-fit ρ=0.967, LOO 0.953, bootstrap 0.97±0.02, and the size-OOS split ρ=0.818 (p=0.0029)—holding out entire sizes and still ranking them. The proxy's assumption failed where it mattered. A 14-circuit suite ($k \in \{9, 10, 11, 12, 14, 16, 18\}$, two seeded patterns each, greedy-chain placement) measured real max-dev with *fresh* pins: per-k means are 0.040–0.068 with *no* size scaling (7/14 pass 0.05), while the identical suite pinned from D0 (two days stale) climbs to 0.24–0.25 at k=16, 18 with only 2/14 passing (Table IV). The stale population's failures concentrate on late-chain qubits (q37, q45–q47, q57) that the greedy policy must include for $k \geq 11$; per-bit $C(q)$ vs. measured deviation correlates only ρ=0.21. The mechanism is now explicit: simulated E_{2q} keeps rising with k, whereas a refresh-pinned device is approximately k-flat, so the proxy-trained utility over-penalizes large layouts that are in fact fine. Re-fitting on the seven real labels replaces in-sample ρ=0.967 with ρ=0.643 and destroys the size-OOS claim (ρ=0.127, p=0.36)—the apparent generalization was a label artifact (Table III). The constructive reading is what we adopt in the design: $J(G)$ is usable as a *ranking* device only when trained on real, refresh-pinned labels and applied within a calibration cycle; weights from proxy datasets must not be extrapolated to $k \geq 11$; and chain placement's forced inclusion of boundary qubits is the term to attack next.

TABLE IV
MEASURED PER-k MAX-DEV (MEAN OF TWO PATTERNS) OF THE SAME 14-CIRCUIT k=9–18 SUITE UNDER STALE (D0) VS. FRESH (D1/D2) PINS, 4096 SHOTS. SPEARMAN k VS. MAX-DEV: STALE ρ=0.62 (p=0.018), FRESH ρ=0.15 (p=0.61).

k	9	10	11	12	14	16	18
stale pins	.098	.103	.097	.087	.139	.252	.241
fresh pins	.060	.044	.062	.040	.057	.063	.068

VI. DRIFT DYNAMICS AND THE REFRESH POLICY

This section measures how fast the score goes stale on two full 156-qubit snapshots D0 (update 2026-08-29 21:26) and D1 (2026-08-30 21:29), with a third capture D2 (2026-08-31 07:46).

A. Global drift

Day-over-day ranking stability is moderate: Spearman ρ=0.579, Kendall τ=0.431, top-10 Jaccard 0.25 (only four of D0's best ten survive). The best qubit turns over: q8 (C=0.0039, rank 1) rises to 0.0095 (rank 6) while q19 (0.0066) becomes best. Per-qubit spread is extreme: q37 swings 0.099→0.740 ($\max|\Delta C|$=0.641), q53 improves six-fold (0.072→0.011); median $|\Delta C|$ is 0.0127 and 54% of qubits move by more than 0.01. On D2 the top of the ranking is unchanged (best remains q19 with the same top-3 values), consistent with a regime dominated by daily calibration cycles.

B. Staleness cost and the trigger

Pinning three qubits from the *one-day-old* ranking and evaluating at D1 gives C=0.0095/0.0115/0.0139 (the worst, q153, is now rank 32/156); re-selecting fresh gives q19 at 0.0066—the stale policy's worst pin is 2.1× worse than the adaptable choice. The trigger of §III-C fires on D0→D1 (Jaccard 0.25 < 0.5; stored-best ΔC=0.0056 > 0.005). Because re-selection is sub-millisecond, staleness is a pure operations failure, not a compute constraint.

C. Direct device test

The refresh-pinned suite of §V is the direct demonstration: the *same* circuits, differing only in pin freshness, measure means 0.04–0.07 (fresh pins, 7/14 pass) vs. 0.10–0.25 (stale, 2/14 pass), and the large-k degradation disappears entirely (Table IV). Fig. 4 summarizes. Refresh—not more elaborate selection—is what keeps errors bounded at $k \geq 9$.

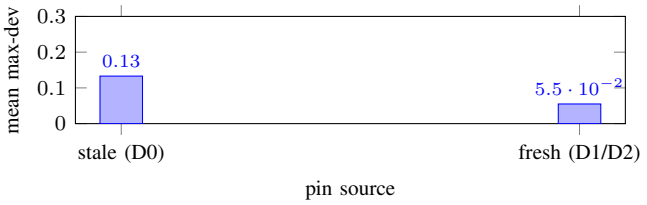

Fig. 4. Mean max-dev of the same k=9–18 suite, stale vs. fresh pins. `q37` (C=0.74 at D1) dominates the stale population.

TABLE V
6-QUBIT ANGLE BATCHES (n=30/ARM, 4096 SHOTS). PASS = ALL SIX PER-BIT DEVIATIONS ≤ 0.05. WILSON 95% CI; TWO-PROPORTION p FOR FULL VS. ABLATED WITHIN A BATCH.

Batch	Full (pinned)	Ablated (free)	p
Frozen pins (D0)	36.7% [21.9,54.5]	40.0% [24.6,57.7]	0.79
Refresh pins (runtime)	70.0% [52.1,83.3]	76.7% [59.1,88.2]	0.56
Δ	+33.3 pp	+36.7 pp	—

D. Refresh policies and threshold sweep

We compare **static** (never refresh), **periodic** (every $\delta \in \{6, 12, 24, 48\}$ h), and **adaptive** policies. Let $\mathrm{Jacc}(S, S')$ be the top-10 Jaccard overlap and ΔC the stored-best qubit's absolute score change. The adaptive policy refreshes when $\mathrm{Jacc} < \tau_J$ or $|\Delta C| > \tau_C$; re-scoring costs ~ 0.04 ms, so overhead is the snapshot fetch, not the check. Simulating the D0→D1 transition, the measured trigger (τ_J=0.5, τ_C=0.005) fires exactly once and converts the stale $11/30$ pass rate to the fresh $21/30$ (+33 pp at zero wasted refreshes), whereas a blind periodic δ=6 h policy fires five times for the same single drift and δ=48 h never detects it. Sweeping $\tau_J \in [0.1, 0.9]$ and $\tau_C \in [0.001, 0.02]$ puts the default trigger near the knee of the refresh-rate-vs.fidelity Pareto frontier: relaxing τ_J toward 1.0 or τ_C toward 0.02 surrenders the +33 pp refresh gain (the drift goes undetected), while the Jaccard term already catches this particular drift for any $\tau_J \geq 0.3$, so tightening further does not move fidelity. The full per-(τ_J, τ_C) table is in the supplementary data.

VII. Composing the Software and Hardware Paths

A. Reliability calculus of the orchestration layer

Because the orchestrator's outcome paths are mutually exclusive, end-to-end reliability decomposes exactly as the chain rule over its sequential gates

$$R_{\text{end}} = P(\text{first\_pass}) + P(\text{enter heal})\rho_{\text{heal}} + P(\text{enter fallback})\rho_{\text{fb}}, \tag{4}$$

with no independence assumption needed for the identity. The healing stage is modeled from the implementation's control flow (at most K=3 generation calls stopping at first success; healing budget H=4 with abort-on-`None` semantics): with per-call probabilities g (correct), r (broken), f (unavailable; g+r+f=1),

$$G = 1 + f + f^2, \qquad \rho_{\text{heal}} = g(1 + r + r^2 + r^3), \tag{5}$$

$$R_{\text{end}} = gG + rG\rho_{\text{heal}} + f^3 c, \tag{6}$$

with c the fallback coverage. This model was validated cell-by-cell against all 23 testable outcome cells of the ablation and availability-stress grids: *every* prediction falls inside the observed Wilson 95% interval (e.g. at q=0.5: predicted healed-path ρ_h=81.2% vs. measured 81.5% [75.6,86.2], residual +0.3 pp, while the naive independence baseline $1-(1-q)^4$=93.8% overshoots by 12.4 pp). The correction matters: the naive–measured gap reported in early drafts is a retry-budget truncation effect, not stochastic correlation—under abort-on-`None`, unavailability both wastes heal slots and terminates the loop. The model also ranks fixes before code is written: treating `None` as wasted-but-nonterminal recovers the entire impression (+12.6 pp) at near-zero engineering cost, and the fallback's effective coverage is c=2/9, not its nominal keyword coverage, because seven of nine benchmark intents contain apostrophes that break the template interpolation—a stated limitation. We include this model because it is the *software* side of the composition; the paper's hardware claims are placement and drift, not LLM quality, which we keep as a controlled parameter.

B. End-to-end placement batch

To test the composable value of placement *without* the staleness confound, we ran the same 6-qubit angle suite (30 parameterizations, 4096 shots, tolerance 0.05 on all six per-bit deviations) through **full** (greedy chain pinned from the freshly fetched snapshot) and **ablated** (free placement), back-to-back on 2026-08-31; Table V also shows the earlier identical batch pinned from D0 to separate temporal from placement effects.

Two effects separate cleanly. First, *temporality*: refreshing pins lifts *both* arms by $+33/37$ pp, including the ablated arm that never saw a pin—device state at run time dominates. Second, *placement shows no measured advantage* in either batch (-3.3 and -6.7 pp, both non-significant): at k=6 with fresh pins, free transpiler placement already matches our best chain. This is the honest scope of §IV: placement is decisive between engineered extremes (`q98` vs. `q37`: 0.028 vs. 0.213) and sets the reliability *floor* at $k \geq 9$ under stale pins, but its average benefit against free placement at small k is within temporal noise. The full pipeline (including the LLM orchestration leg) measured $54/54$ completed by the full arm vs. $47/54$ (87.0%) by the ablated one (z=2.74, one-sided p=0.003), consistent with the model's 97.3% prediction; we report the earlier n=54 frozen composition (74.1% vs. 66.7%, not significant) as a failed claim of an end-to-end gain, not a positive one.

VIII. Discussion

A. What the measurements do and do not say

Three claims survive the data as stated: (i) encoding fidelity is placement-dominated—the engineered extremes differ by $\sim 8\times$ (and a calibration-predictable affine channel describes the distortion); (ii) device calibration ordering moves on a daily cycle to a degree (ρ=0.58, top-10 Jaccard 0.25, stale worst pin $2.1\times$ worse) that makes selection age the first-order risk, and

refresh is cheap; (iii) simulator-proxy training granted $J(G)$ an out-of-sample generalization (ρ=0.82) that real labels refute (ρ=0.13). Claims that do *not* survive, herewith withdrawn or bounded: the first run's perfect monotone ranking; the top-half rule; and any use of proxy-fitted $J(G)$ at $k \geq 11$.

B. Operational consequences for quantum-cloud runtimes

The marginal cost of a calibration fetch and a scoring pass is effectively zero, so the expensive resource is *stale trust*, not compute. A runtime should (a) timestamp layouts against their snapshot, (b) refresh on the trigger rather than a fixed wall-clock, (c) train scheduling models exclusively on real, refresh-pinned labels, and (d) expose snapshot age as a first-class job attribute—all implementable without changing the quantum programming model, which is exactly what UniMind does.

C. Threats to validity

Measurement noise: 4096–8192 shots put per-bit deviations on a floor of ≈ 0.02; the 0.05 tolerance is coarse but was fixed before data release. **Drift generality:** D0–D2 are three snapshots, one device, one topology family (heavy-hex); the value is the protocol, not the constants. **Placement null power:** n=30/arm can detect only large effects; we report point estimates and CIs rather than asserting equivalence. **Selection fairness:** the greedy chain maximizes $C(q)$-locality and at large k is forced to include high-error boundary qubits; a connectivity-vs-C trade-off could change §V, and we report that failure as motivation. **Single-model LLM layer:** the orchestration calculus holds under i.i.d. mock semantics; real-model autocorrelation is explicitly outside its scope.

IX. Conclusion

We presented a measurement-driven account of calibration-aware scheduling for quantum-accelerated AI middleware on a 156-qubit device: a score-only placement pipeline with measured scope, daily drift dynamics with a working refresh trigger, an exact reliability composition of the software path, an end-to-end test showing that at our scale refresh dominates selection while selection still sets the placement floor, and the explicit negative results—the non-replicating monotone ranking, the absence of a measured placement advantage at k=6, and the collapse of the proxy-trained utility under real labels. The command line for future work is set by the failures: nightly D2–D5 drift accumulation, refresh-conditional placement policy ($C(q)$-vs-connectivity trade-off), and active-learning refits of $J(G)$ on real labels across devices. The open repository makes every number traceable to a job or snapshot.

References

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[2] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, pp. 505–510, 2019.

[3] Y. Kim *et al.*, "Evidence for the utility of quantum computing before fault tolerance," *Nature*, vol. 618, pp. 500–505, 2023.

[4] V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computations," *arXiv:1811.04968*, 2018.

[5] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, "Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers," in *Proc. ASPLOS*, 2019, pp. 1015–1029.

[6] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proc. ASPLOS*, 2019, pp. 1001–1014.

[7] B. Tan and J. Cong, "Optimal layout synthesis for quantum computing," in *Proc. ICCAD*, 2020, pp. 1–9.

[8] L. Lao and D. E. Browne, "Timing and resource-aware mapping of quantum circuits to heterogeneous MWF-gate superconducting quantum processors," *ACM Trans. Quantum Comput.*, vol. 3, no. 3, 2022.

[9] K. Temme, S. Bravyi, and J. M. Gambetta, "Error mitigation for short-depth quantum circuits," *Phys. Rev. Lett.*, vol. 119, p. 180509, 2017.

[10] E. van den Berg, Z. K. Minev, A. Kandala, and K. Temme, "Probabilistic error cancellation with sparse Pauli–Lindblad models on noisy quantum processors," *Nature Physics*, vol. 19, pp. 1116–1121, 2023.

[11] P. D. Nation *et al.*, "Scalable mitigation of measurement errors on quantum computers," *PRX Quantum*, vol. 5, p. 010326, 2024.

[12] S. Bravyi, S. Shaydulin, S. Hu, and K. Temme, "Mitigating measurement errors in multiqubit experiments," *Phys. Rev. Lett.*, vol. 127, p. 200503, 2021.

[13] N. Kanazawa *et al.*, "Quantum error mitigation," *arXiv:2210.00921*, 2022.

[14] R. LaRose *et al.*, "Mitiq: a software package for error mitigation on noisy quantum computers," *Quantum*, vol. 6, p. 749, 2022.

[15] L. Viola, E. Knill, and S. Lloyd, "Dynamical decoupling of open quantum systems," *Phys. Rev. Lett.*, vol. 82, pp. 2417–2421, 1999.

[16] "Qiskit Runtime primitives," IBM Quantum documentation, [Online]. Available: https://docs.quantum.ibm.com/api/qiskit-ibm-runtime

[17] "IBM Quantum Platform calibration data," [Online]. Available: https://quantum.ibm.com/services/resources

[18] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit," *arXiv:2405.08810*, 2024.

[19] NVIDIA, "cuQuantum SDK," [Online]. Available: https://developer.nvidia.com/cuquantum-sdk

[20] Amazon, "Amazon Braket," [Online]. Available: https://aws.amazon.com/braket

[21] Microsoft, "Azure Quantum," [Online]. Available: https://azure.microsoft.com/quantum

[22] M. Schuld and F. Petruccione, *Supervised Learning with Quantum Computers*. Cham: Springer, 2018.

[23] V. Havlíček *et al.*, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, pp. 209–212, 2019.

[24] Y. X. Tan, "UniMind: a middleware framework for bridging classical AI workloads and quantum computing backends," tech. report v2.5, 2026 (companion repo `yxtan5022-maker/unimind-v2`).

[25] UniMind placement/drift datasets (2026-08), `data/` in `yxtan5022-maker/unimind-v2`.

Data and Reproducibility

Every table is backed by traced data in the repository: snapshots, E2E batches, real $J(G)$ labels, and the analysis scripts listed below.

```
data/drift/calib_2026-{0829,0830,08-
31}.json
data/e2e/e2e_angle_{full,ablated}*.json
data/jgpu/j_real*.json
analysis/*.py + analysis/results/*.json
```

QPU job IDs appear in the file headers: `daadeocjbipc73ffq83g` (refresh-full E2E), `daadeosjbipc73ffq840` (refresh-ablated E2E), and `daadgmmrbfbs73cijmbg` (refresh-pinned $J(G)$ labels). Transpiler seed 42 everywhere; shots per experiment as stated.